

青凤

蒲松龄先生写狐，多寄寓人情。我今日所记的青凤，却并非《聊斋》里那位耿生的心上人，而是一只住在树里的狐女。她有一处庭院，名为“青凤斋”。此院看似寻常，却暗合奇门——无论多少仆从搬入，庭院从不偏斜，始终维持着一种幽静的平衡。

我问青凤：“何以能如此？”

她笑而不答，只是踏着月光，在院中舞了起来。那舞步，竟是一套旋转的法门。

我醒来后，将那套舞步默写成了一段程序。它没有青凤的风姿，但确实能维持一棵树的平衡——这或许便是青凤赠我的唯一信物了。

1 外景：头文件

既是程序，总得有一角天空。这些头文件便是我的“外景”。

@ 外景 #

```
#include <stdio.h>
#include <stdlib.h>
=> 入梦 <9>
```

1

2 庭院深深：结点之形

青凤斋中，每一名仆从皆有编号，且知晓自己所在位置的高下。结点便是这样一名仆从。

@ 庭院 #

```
typedef struct Node {
    int key;
    struct Node *left;
    struct Node *right;
    int height;
} Node;
=> 入梦 <9>
```

2

3 步法根基：求高与取大

舞步之前，先须知道脚下高低。青凤用下面两个动作来感知。

@ 求高与取大 #

```
int height(Node *n) {
    if (n == NULL) return 0;
    return n->height;
}

int max(int a, int b) {
    return (a > b) ? a : b;
}
=> 入梦 <9>
```

3

4 月下之舞：右旋

若庭院西北角偏重，青凤便以右足为轴，轻轻一引，将重负移至东南。此为右旋。

@ 右旋 #

4

```
Node *rotate_right(Node *y) {
    Node *x = y->left;
    Node *T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
}
=> 入梦 <9>
```

5 月下之舞：左旋

若东南过沉，她便左足一旋，将多余分量移至西北。

@ 左旋 #

5

```
Node *rotate_left(Node *x) {
    Node *y = x->right;
    Node *T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}
=> 入梦 <9>
```

6 新仆入门：插入结点

每有新仆来投，青凤便依其编号，安置于庭院某处。安置毕，她必踏一步法，检查四角是否平衡。若失衡，便旋舞一二，复归中正。

@ 新仆入门 #

6

```
Node *insert(Node *node, int key) {
    if (node == NULL) {
        Node *new_node = (Node *)malloc(sizeof(Node));
        new_node->key = key;
        new_node->left = NULL;
    }
}
```

```

        new_node->right = NULL;
        new_node->height = 1;
        return new_node;
    }

    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);
    else
        return node;

    node->height = 1 + max(height(node->left), height(node->right));

    int balance = height(node->left) - height(node->right);

    // 左左偏重, 右旋
    if (balance > 1 && key < node->left->key)
        return rotate_right(node);

    // 右右偏重, 左旋
    if (balance < -1 && key > node->right->key)
        return rotate_left(node);

    // 左右偏重, 先左旋再右旋
    if (balance > 1 && key > node->left->key) {
        node->left = rotate_left(node->left);
        return rotate_right(node);
    }

    // 右左偏重, 先右旋再左旋
    if (balance < -1 && key < node->right->key) {
        node->right = rotate_right(node->right);
        return rotate_left(node);
    }

    return node;
}
=> 入梦 <9>

```

那“左右偏重”“右左偏重”两段，正是青凤舞中最为繁复的连环步。她旋动时，衣袖交叠如飞花，旁人只见其影，不见其形。

7 庭院漫步：中序遍历

若想遍访青凤斋中所有仆从，只需沿一条幽径依次行去。那顺序，竟是从小到大，恰如展开一卷花名册。

```
void inorder(Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->key);
        inorder(root->right);
    }
}
=> 入梦 <9>
```

8 曲终人散：释放庭院

故事终有尽时。青凤散去，仆从亦各归其位。这片庭院，终须还给荒草。

@ 释放庭院 #

8

```
void free_tree(Node *root) {
    if (root != NULL) {
        free_tree(root->left);
        free_tree(root->right);
        free(root);
    }
}
=> 入梦 <9>
```

9 入梦与醒：主函数

我带着梦中所记，将青凤斋建在一段小小的主函数里。请来了六位仆从，依次编号为30、20、40、10、25、35。他们入门后，我沿中庭信步，果然看到了从小到大的序列。那正是青凤舞后的太平景象。

@ 入梦 #

9

```
# 外景 @ <1>
# 庭院 @ <2>
# 求高与取大 @ <3>
# 右旋 @ <4>
# 左旋 @ <5>
# 新仆入门 @ <6>
# 庭院漫步 @ <7>
# 释放庭院 @ <8>
```

```
int main() {
    Node *root = NULL;
    int keys[] = {30, 20, 40, 10, 25, 35};
    int n = sizeof(keys) / sizeof(keys[0]);

    for (int i = 0; i < n; i++) {
        root = insert(root, keys[i]);
    }

    printf("青凤斋中仆从序列: ");
```

```
    inorder(root);  
    printf("\n");  
  
    free_tree(root);  
    return 0;  
}
```

10 后记

青凤从未告诉我她是否只是一段代码。我只知道，每当运行这个程序，那片月下庭院便会重现一次。树影婆娑，仆从进退，平衡之舞永不歇息。或许这就是蒲留仙所谓的“异史氏曰”：物之恒者，必有灵焉。